

下面重点介绍 **GLM-5 的训练方法**。相比模型结构，GLM-5 真正的突破其实更多来自**训练流程设计**：它不是简单扩大参数，而是围绕 **长上下文 Agent 能力、代码能力、RL 后训练效率**重新设计了一套 pipeline。

整体可以概括为：



论文 Figure 5 给出了完整训练流程。

1. 总体训练规模

GLM-5 Base Model:

阶段	数据量
Base pretraining	27T tokens
Mid-training	约1.5T+ tokens
DSA adaptation	20B tokens
总训练token	28.5T tokens

论文说明：

GLM-5 的 base model training 从 27 trillion token corpus 开始，最终所有训练阶段累计达到 28.5T tokens。

2. 第一阶段：Base Pretraining

目标：

训练一个具备：

- 通用语言能力
- 代码能力
- 推理能力

的基础模型。

数据组成：

Web

增强：

- 高质量网页
- Wikipedia知识
- 长尾知识

Code

重点加强：

- Github代码
- 软件工程数据
- 多语言代码

论文：

Code pre-training corpus 增加28%的去重token，并增强低资源语言采样质量。

Math & Science

包括：

- 数学
 - 科学论文
 - 教育数据
-

3. 第二阶段：Mid-training（非常关键）

GLM-5 不直接从短上下文训练到200K。

而是：

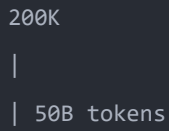
Progressive Context Scaling





128K
|
| 500B tokens
↓

Stage 3



200K
|
| 50B tokens

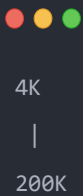
论文明确：

Progressive extend context window across three stages: 32K, 128K, and 200K.

为什么需要 Mid-training?

因为：

直接：



4K
|
200K

会导致：

- attention分布变化
- RoPE失效
- 长距离依赖能力差

所以采用：



短context学习能力

↓

中context适应

↓

超长context强化

4. 长上下文数据训练

GLM-5 特别强调：

不是简单增加长度。

而是构造：

Long-context Agent Data

包括：

1. Repo级代码

例如：



repo

```
|— src/  
|— tests/  
|— README  
|— issue  
|— PR
```

拼接成：



200K token sequence

让模型学习：
整个软件工程过程。

2. Synthetic Agent Trajectory

例如：



这些轨迹用于训练：
长期规划能力。

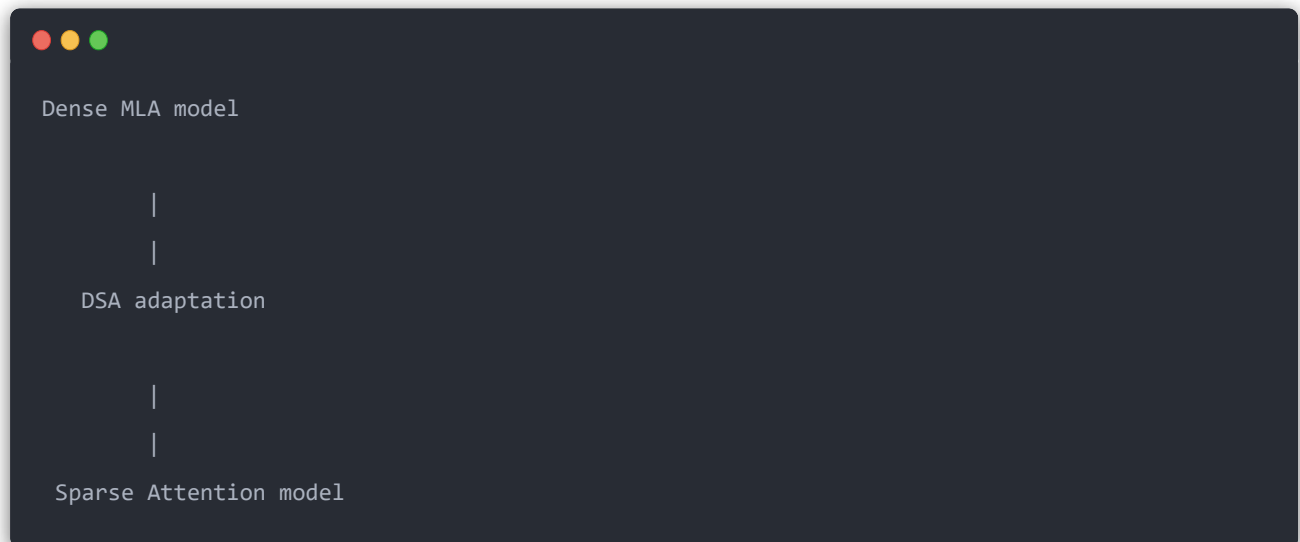
5. DSA Continual Training

这是 GLM-5 非常重要的一步。

注意：

它不是重新训练一个DSA模型。

而是：



论文称：

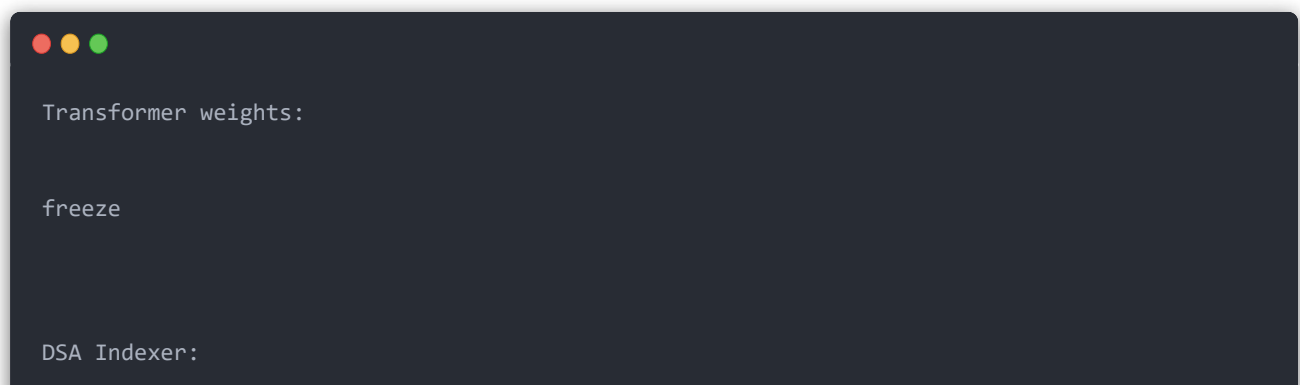
DSA training begins from the base model at the end of mid-training.

分两个阶段：

Stage 1: Indexer Warmup

1000 steps:

冻结模型参数：



```
train
```

学习：

哪些token重要。

Stage 2: Sparse adaptation

继续：

20B tokens

同时训练：



```
graph TD; A[Attention] -- "+" --> B[Indexer]
```

最终：

DSA性能接近原MLA。

6. SFT阶段

目标：

把 Base Model 变成 Assistant。

数据三类：

1. General Chat

例如：

- QA

- 写作
 - 翻译
-

2. Reasoning

包括：

数学

科学

代码推理

3. Coding & Agent

这是GLM-5重点。

数据：

真实软件工程任务。

例如：



修复

论文：

GLM-5 constructs a large number of execution environments to obtain high-quality trajectories.

7. SFT中的特殊设计：Thinking模式

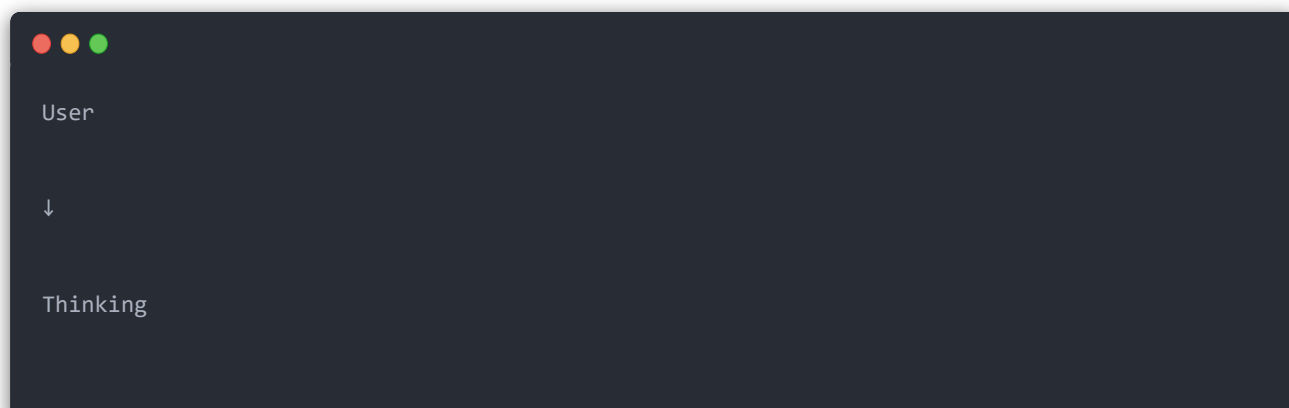
GLM-5支持三种thinking：

1. Interleaved Thinking

传统：



GLM-5：



↓

Tool call

↓

Thinking

↓

Answer

适合Agent。

2. Preserved Thinking

多轮代码任务：

传统：



conversation1

discard reasoning

conversation2

重新推理

GLM-5：

保存之前thinking。



历史reasoning

+

新任务

↓

继续推理

3. Turn-level Thinking

用户控制：



简单问题：

关闭thinking

复杂任务：

开启thinking

8. RL阶段：三阶段强化学习

这是 GLM-5 最大训练创新之一。

整体：



SFT model

↓

Reasoning RL

↓

Agentic RL

↓

General RL

↓

Cross-stage Distillation

论文：

sequential Reinforcement Learning pipeline — starting with Reasoning RL, followed by Agentic RL, and finishing with General RL.

9. Reasoning RL

目标：

增强：

- 数学
- 科学
- 编程

算法：

基于：

GRPO

但是加入：

IcePop

解决：

training policy

和

inference policy

不一致。

公式：

$$\rho = \frac{\pi_{train}}{\pi_{infer}}$$

如果：

偏差太大：

discard。

10. Agentic RL (最重要)

因为 GLM-5 定位：

Agent Engineer。

普通RL：



Agent RL:



任务



规划



调用工具



修改代码



运行环境



得到reward

问题：

Agent rollout 很慢。

例如：

一个coding任务：



10分钟

另一个：



10秒

同步RL：

必须等最长。

GPU大量空闲。

11. 异步RL架构

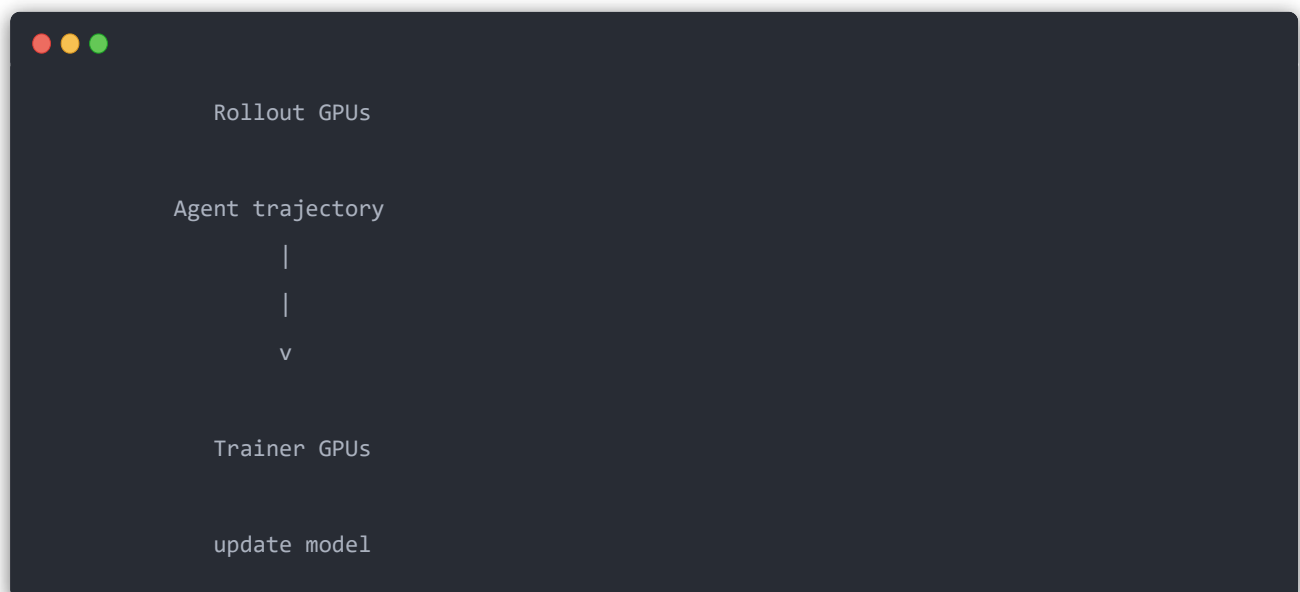
GLM-5提出：

Training 和 Rollout 分离

传统：



GLM-5：



两个独立运行。

论文：

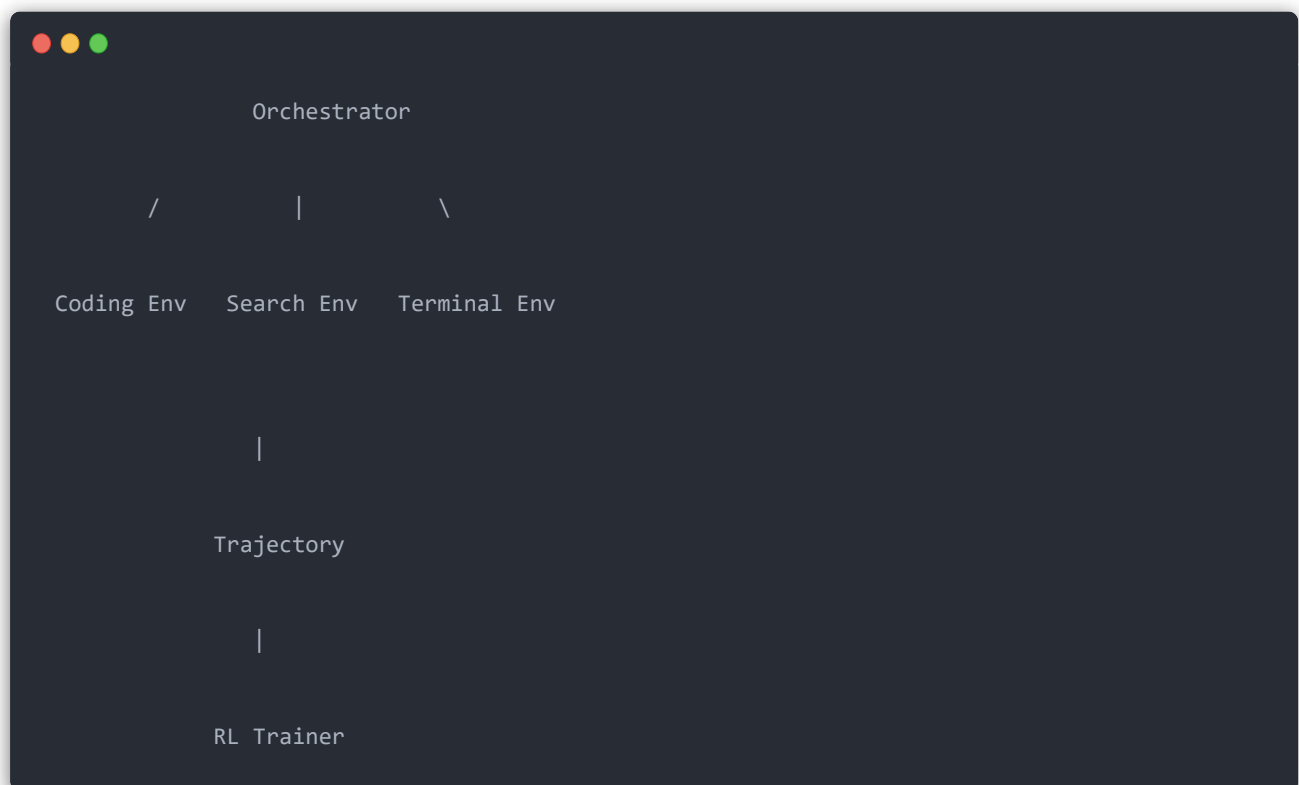
decouple inference and training engines onto different GPU devices.

12. Multi-task Rollout Orchestrator

为了支持：

- Coding Agent
- Search Agent
- Terminal Agent

设计：



特点：

- 超过1000并发rollout
- 动态任务比例调整

13. TITO: Token-in-Token-out

这是异步RL非常关键。

传统：



问题：

token可能不一致。

GLM-5：

直接：



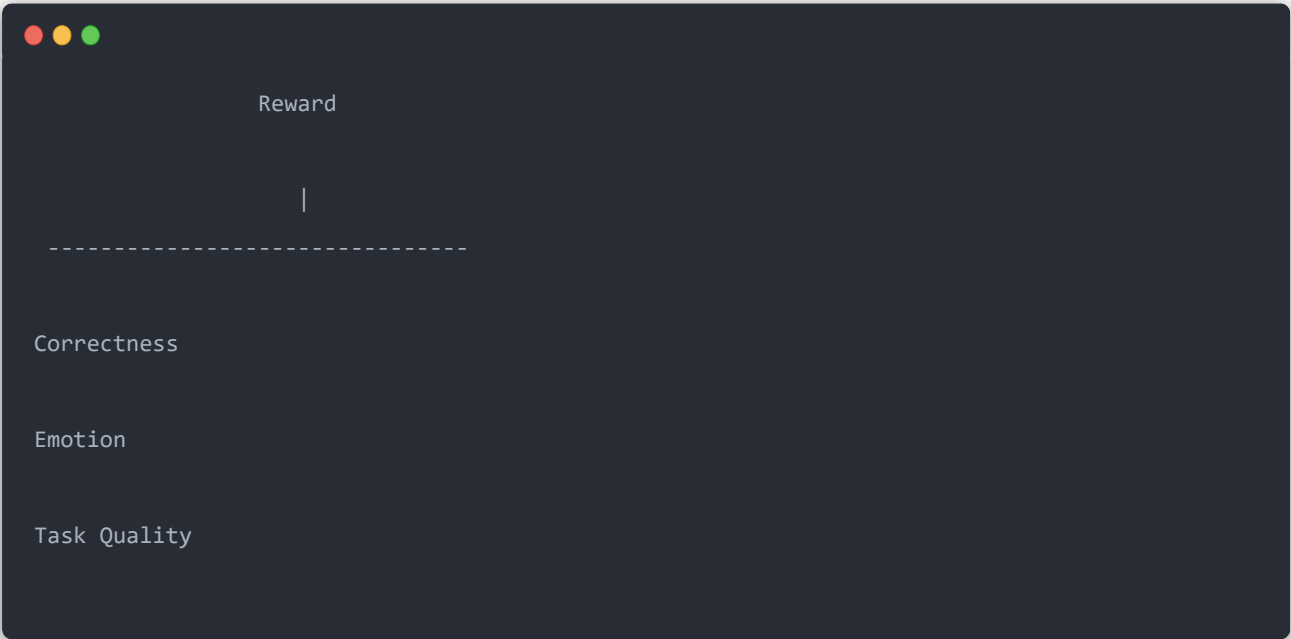
避免：

- tokenizer mismatch
- action alignment错误

14. General RL

最后做人类偏好优化。

三个reward：



奖励来源：

类型	特点
Rule reward	准确
ORM	稳定
GRM	泛化

三者组合。

15. Cross-stage Distillation (非常关键)

问题：

连续RL：



可能：

后面的训练破坏前面的能力。

例如：

Agent能力增强：

数学能力下降。

解决：

On-policy Cross-stage Distillation

最后：

让：

之前阶段模型

作为teacher。





恢复能力。

论文：

prevent catastrophic forgetting while becoming a robust generalist.

总结：GLM-5训练创新点

模块	创新
Pretrain	27T token, 代码优先
Mid-training	32K→128K→200K
Long context	Repo + Agent trajectory
DSA training	Dense→Sparse继续训练
SFT	Agent trajectory + thinking
Reasoning RL	GRPO+IcePop
Agent RL	异步RL
Infrastructure	Rollout/Training解耦
Stability	TITO + importance sampling
Alignment	Cross-stage distillation